# GRADIQ – STUDENT PERFORMANCE PREDICTION SYSTEM

A PROJECT REPORT

SUBMITTED BY

| Roll Number | Registration Number | Student Code | Student Name |
|---|---|---|---|
| 24103460102 | 24103002884 | BWU/BCA/24/107 | Sourav kr singh |

***Of in partial fulfilment for the award of the degree***

**BACHELOR OF COMPUTER APPLICATION**

IN

***DEPARTMENT OF COMPUTATIONAL SCIENCE***




**BRAINWARE UNIVERSITY**

**398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125**

# INDEX


| SI | Content | Page No. |
|---|---|---|
| 1 | ABSTRACT | |
| 2 | INTRODUCTION | 1 |
| 3 | LITERATURE SURVEY | 2 |
| 4 | SYSTEM ANALYSIS AND DESIGN | 3 |
| 5 | METHODOLOGY | 4 |
| 6 | IMPLEMENTATION | 7 |
| 7 | TESTING AND DEBUGGING | |
| 8 | CONCLUSION | |
| 9 | FUTURE ENHANCEMENT | |
| 10 | REFERENCES | |

# ABSTRACT

Gradiq is a web-based Student Performance Prediction System developed to simplify academic performance management and support data-driven analysis of student performance. The primary objective of this project is to provide an integrated platform where academic records can be managed, analysed, and used to predict student performance through machine learning techniques.

The system considers various academic parameters, including Assignment 1, Assignment 2, Class Test 1, Class Test 2, Presentation Marks, and Attendance. These parameters are processed to calculate internal assessment performance and prepare the relevant input features for analysis and prediction. The system also provides facilities for student record management, academic data handling, data visualization, and performance monitoring through an interactive dashboard.

Gradiq follows a machine learning workflow that includes data validation, preprocessing, exploratory data analysis, feature preparation, model training, and performance evaluation. Multiple regression algorithms, including Linear Regression, Decision Tree Regressor, and Random Forest Regressor, are used to analyse academic data and evaluate predictive performance. Evaluation metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and $R^2$ Score are used to assess model performance.

The application is developed using React and Vite for the frontend, Python and Flask for the backend, and TiDB Cloud for database management. The machine learning functionality is integrated into the backend through dedicated APIs, while the frontend provides an accessible interface for administrators to manage student information, view analytical results, and generate performance predictions.

Gradiq aims to support educational institutions in understanding student performance patterns and identifying students who may require additional academic support. The system provides a centralized and scalable platform that combines student data management, analytical visualization, and machine learning-based performance prediction in a single application.

# INTRODUCTION

Education plays a vital role in the development of individuals and society. Academic performance is an important indicator of a student's learning progress, knowledge, and understanding. Educational institutions regularly collect academic information through assignments, class tests, presentations, attendance, and semester examinations. Analysing this information can help institutions understand students' academic progress and identify areas where improvement is required.

With the advancement of technology, Machine Learning (ML) and data analytics have become valuable tools in the education sector. These technologies enable educational institutions to process academic records, identify performance patterns, and generate useful insights from student data. Educational Data Mining (EDM) and predictive analytics provide opportunities to make academic monitoring more systematic and data-driven.

## 1.1 Problem Statement

Traditional student performance monitoring often involves manually maintaining academic records and evaluating students through individual assessments and final examinations. Managing these records can become time-consuming and difficult, especially when dealing with a large number of students. Furthermore, analysing academic performance manually may not provide sufficient insights into individual student progress or identify students who need additional academic support at an early stage.

There is a need for a centralized and efficient system that can manage student information, organize academic assessment records, analyse performance-related data, and support performance prediction. Such a system can reduce manual effort, improve data accessibility, and help administrators make informed academic decisions.

## 1.2 Proposed System

To address these challenges, this project introduces **Gradiq – Student Performance Prediction System**, a web-based application that integrates student data management, academic performance analysis, and machine learning-based prediction.

The system provides an interactive platform where administrators can manage student records and maintain academic assessment information. It considers parameters such as Assignment 1, Assignment 2, Class Test 1, Class Test 2, Presentation Marks, and Attendance for academic performance analysis.

Gradiq also incorporates a machine learning pipeline that supports data preprocessing, exploratory data analysis, model training, and performance evaluation. Regression algorithms such as Linear Regression, Decision Tree

Regressor, and Random Forest Regressor are used to analyse academic data and generate predictions. The system presents relevant information through dashboards, charts, and analytical results, making student performance easier to monitor and understand.

## 1.3 Objectives of the Project

The main objective of Gradiq is to develop a centralized web-based platform for managing student academic information and supporting performance analysis. The system provides facilities for adding, updating, viewing, and managing student records while organizing assessment details such as assignments, class tests, presentations, and attendance. It also aims to perform data preprocessing and exploratory data analysis, implement machine learning algorithms for student performance prediction, evaluate models using standard performance metrics, and present academic information through interactive dashboards and visualizations. Another important objective is to support the identification of students who may require additional academic attention.

## 1.4 Scope and Significance of the Project

Gradiq is designed to simplify academic record management and provide analytical support for educational institutions. It combines student information management with machine learning capabilities in a single web-based application.

The system can help administrators maintain organized academic records, monitor student performance, and understand patterns in assessment data. Its analytical and predictive capabilities can support early identification of students who may be at risk of underperforming, allowing institutions to consider appropriate academic guidance and improvement strategies.

The project also demonstrates the practical integration of frontend development, backend API development, database management, and machine learning. Its modular architecture provides a foundation for future improvements, including additional analytical features, larger datasets, advanced prediction techniques, and student-focused academic recommendations.

## 1.5 Organization of the Report

This report is organized into five main chapters. Chapter 1 presents the project background, problem statement, proposed system, objectives, scope, and significance. Chapter 2 discusses existing research, concepts, and techniques related to educational data mining and student performance prediction. Chapter 3 explains the system requirements, architecture, database design, data preprocessing, machine learning workflow, and model evaluation methods. Chapter 4 describes the implementation of the Gradiq application, its major modules, testing procedures, and analytical results. Finally, Chapter 5 summarizes the project, discusses its outcomes and limitations, and presents

possible future enhancements.

## 2. LITERATURE SURVEY

Several researchers have explored the application of data mining and machine learning techniques to analyse and predict student academic performance. Educational Data Mining (EDM) has emerged as an important research area that focuses on extracting meaningful patterns from educational data to improve academic monitoring, student evaluation, and learning outcomes.

Cortez and Silva (2008) conducted research on student performance prediction using data mining techniques. Their study demonstrated how academic and demographic attributes could be analysed to understand factors associated with student achievement. The research highlighted the importance of historical academic information in predicting student performance and provided a foundation for further studies involving machine learning in education. Their work established the significance of using data-driven techniques to support academic decision-making.

Romero and Ventura (2010) presented a comprehensive review of Educational Data Mining and discussed its applications in educational environments. They identified several important areas, including prediction, clustering, relationship mining, and visualization. Their research explained how data mining techniques can help educational institutions understand student behaviour, analyse academic patterns, and identify students who may require additional academic support.

Kotsiantis et al. (2004) investigated the use of machine learning techniques for predicting student performance in distance learning environments. Their study demonstrated the usefulness of analysing multiple student-related attributes to identify patterns associated with academic outcomes. It also highlighted the potential of predictive models to support educational planning and improve student monitoring.

Al-Barrak and Al-Razgan (2016) explored the application of decision tree techniques for analysing and predicting student academic performance. Their research demonstrated how interpretable machine learning models can identify important factors associated with academic achievement. Such approaches are useful in educational environments because they can provide understandable results that support academic evaluation and decision-making.

Hussain et al. (2018) examined the application of different machine learning approaches in educational data analysis. Their research highlighted the importance of

comparing multiple algorithms to identify suitable predictive models. Ensemble learning techniques, such as Random Forest, are particularly useful for analysing complex relationships among different academic attributes and can provide an alternative to traditional statistical approaches.

Yassein et al. (2017) also contributed to research on machine learning applications in student performance prediction. Their work emphasized the importance of selecting suitable predictive techniques and evaluating model performance using appropriate evaluation measures. Such research provides a basis for comparing regression algorithms when the objective is to estimate numerical academic performance values.

Based on these existing studies, it can be observed that machine learning and educational data analysis have significant potential for improving academic performance monitoring. However, managing student records, organizing assessment information, analysing academic data, and presenting predictive results through a unified application remain important practical requirements. A centralized system can make these activities more accessible and systematic for educational administrators.

The present project, **Gradiq – Student Performance Prediction System**, builds upon these concepts by integrating student data management, academic assessment tracking, exploratory data analysis, and machine learning-based prediction into a single web-based platform. The system uses academic parameters such as assignment marks, class test marks, presentation marks, and attendance to support performance analysis. It implements regression algorithms, including Linear Regression, Decision Tree Regressor, and Random Forest Regressor, and evaluates their performance using MAE, MSE, RMSE, and $R^2$ Score. The application combines these analytical capabilities with a React-based frontend, Flask backend, and TiDB Cloud database to provide a centralized environment for managing and analysing student academic information.

## 3. SYSTEM ANALYSIS AND DESIGN

# 3.1 System Analysis

System analysis is the process of examining the existing approach, identifying its limitations, and defining the requirements of the proposed system. Gradiq is designed to improve student academic record management and provide machine learning-based academic performance analysis through a centralized web application.

# 3.1.1 Existing System

In many educational institutions, student academic records are maintained using spreadsheets, registers, or separate software applications. Assessment details such as assignments, class tests, presentations, attendance, and semester marks are often managed independently. This makes it difficult to maintain centralized records and analyse the overall academic performance of students efficiently.

Traditional evaluation methods mainly focus on individual assessments and final examination results. They may not provide sufficient analytical insights into student performance patterns or help administrators identify students who require additional academic attention. Manual record management also increases administrative effort and the possibility of data entry errors.

## 3.1.2 Proposed System

The proposed system, **Gradiq – Student Performance Prediction System**, is a web-based application developed to simplify student data management and support academic performance analysis using machine learning techniques.

The application provides a centralized platform where administrators can manage student information, maintain academic assessment records, and monitor student performance through an interactive dashboard. It considers assessment parameters such as Assignment 1, Assignment 2, Class Test 1, Class Test 2, Presentation Marks, and Attendance.

The system also integrates a machine learning pipeline for data preprocessing, exploratory data analysis, model training, evaluation, and performance prediction. Linear Regression, Decision Tree Regressor, and Random Forest Regressor are used to analyse academic data and generate predictions. The results are presented through analytical charts and performance indicators to support better academic monitoring and decision-making.

## 3.1.3 Feasibility Study

A feasibility study determines whether the proposed system can be developed and operated effectively with the available technical resources, infrastructure, and operational requirements. The feasibility of Gradiq is evaluated in terms of technical, operational, and economic aspects.

## 3.1.4 Technical Feasibility

Gradiq is developed using modern web technologies and open-source frameworks. React and Vite are used to develop the frontend, while Python and Flask are used for backend development and API integration. TiDB Cloud is used for database management and storing application records.

Machine learning functionalities are implemented using Python-based data processing and machine learning libraries. The application is deployed using Cloudflare Workers for the frontend and Render for the backend. These technologies provide the required infrastructure for developing, maintaining, and accessing the application through a web browser. Therefore, the project is technically feasible.

## 3.1.5 Operational Feasibility

The system provides a user-friendly interface that allows administrators to manage student information and academic records without requiring advanced technical knowledge. Its dashboard provides access to student records, assessment information, analytical results, and prediction-related features.

The centralized structure reduces the need for maintaining multiple separate records and makes academic information easier to access and manage. The web-based deployment also allows authorized users to access the application through an internet connection. Therefore, Gradiq is operationally feasible.

## 3.1.6 Economic Feasibility

The project uses open-source technologies and cloud-based services for application development, database management, and deployment. Frameworks such as React, Flask, and Python reduce software licensing expenses. The use of cloud infrastructure also avoids the need to maintain dedicated physical servers for the application. Therefore, the project can be developed and maintained with relatively low infrastructure and software costs.

## 3.2 Requirement Analysis

Requirement analysis identifies the essential functions and technical conditions necessary for the successful operation of the system. Gradiq requires a frontend interface, backend services, database connectivity, and machine learning components to perform its operations.

### 3.2.1 Functional Requirements

The functional requirements describe the operations that the Gradiq system must perform. The application must provide user authentication and allow authorized users to access the system. It must support student record management, including adding, viewing, updating, and deleting student information.

The system must allow administrators to manage academic assessment information, including assignment marks, class test marks, presentation marks, and attendance. It must validate the entered information and maintain student records in the database.

The application must support the collection and preparation of academic data for machine learning analysis. It must provide data preprocessing, exploratory data analysis, feature preparation, and machine learning model training functionalities. The system must implement and evaluate regression algorithms, including Linear Regression, Decision Tree Regressor, and Random Forest Regressor, using appropriate evaluation metrics such as MAE, MSE, RMSE, and $R^2$ Score.

The system must also provide academic performance predictions through the integrated backend API and display relevant analytical information through dashboards, charts, and performance indicators. It should support the management and validation of training records used by the machine learning pipeline.

### 3.2.2 Non-Functional Requirements

The non-functional requirements define the quality attributes and operational expectations of the application. Gradiq should provide a responsive and user-friendly interface that works across desktop, tablet, and mobile devices. The system should maintain data consistency and provide reliable communication between the frontend, backend, and database.

Authentication and access control should protect application resources from unauthorized access. The backend should validate incoming requests and handle errors appropriately. The application should also provide maintainable and modular code so that additional features and improvements can be introduced in the future.

## 3.3 System Design

System design describes the overall structure of Gradiq, including its architecture, data flow, input and output operations, and interactions between different application components.

## 3.3.1 System Architecture

Gradiq follows a client-server architecture consisting of a frontend, backend, database, and machine learning components.

The frontend is developed using React and Vite and provides the user interface for authentication, student management, academic record handling, dashboards, and analytical visualizations. It communicates with the backend through REST APIs using HTTP requests.

The backend is developed using Python and Flask. It manages application logic, authentication, student-related operations, academic records, machine learning requests, and communication with the database. Dedicated backend routes handle different application functionalities.

TiDB Cloud is used as the database management system for storing user information, student records, predictions, and machine learning training records. The machine learning module processes the relevant academic data, trains regression models, evaluates their performance, and provides prediction results through backend services.

The frontend is deployed on Cloudflare Workers, while the backend is deployed on Render. This architecture separates presentation, application logic, database operations, and machine learning functionality, making the system easier to maintain and extend.

## 3.3.2 System Flowchart

The overall workflow of Gradiq begins when an authorized user accesses the web application through a browser. The user completes the authentication process, after which the system provides access to the appropriate dashboard.

The administrator can manage student information and maintain academic assessment records. When student or assessment information is submitted, the frontend sends the request to the Flask backend through the appropriate API. The backend validates the information and performs the required database operations using TiDB Cloud.

For machine learning operations, the system retrieves eligible academic training records and prepares the data for preprocessing and analysis. The machine learning pipeline performs feature preparation, divides the dataset into training and testing sets, trains the selected regression algorithms, and evaluates their performance using standard evaluation metrics. The selected model can then be used to generate predictions when a trained model is available.

Finally, the backend returns the relevant information to the frontend, where student records, analytical charts, model evaluation results, and prediction outputs are displayed through the dashboard.

## 3.3.3 Input and Output Design

**Input Design:** Gradiq accepts user authentication details, student information, and academic assessment data through its web-based interface. Student-related inputs include the required identification and academic information. Assessment inputs include Assignment 1, Assignment 2, Class Test 1, Class Test 2, Presentation Marks, and Attendance. The system also supports academic training data through its data management and training functionalities.

**Output Design:** The system generates several types of outputs based on user operations. These include student records, dashboard summaries, academic assessment information, data quality reports, exploratory data analysis visualizations, machine learning evaluation metrics, and student performance predictions. Prediction results are displayed through the application interface with appropriate status information to distinguish machine learning predictions from rule-based estimates or cases where a trained model is unavailable.

## 3.4 Tools and Technologies Used

**Frontend Technologies:** React and Vite are used to develop the user interface and manage the frontend application. JavaScript is used for application functionality, while Axios handles communication with backend APIs. Recharts is used for analytical charts and data visualization.

**Backend Technologies:** Python is used as the primary backend programming language, and Flask is used to develop REST APIs and manage application logic.

**Database Technology:** TiDB Cloud is used for database management and storing application data, including user information, student records, prediction records, and machine learning training records.

**Machine Learning and Data Analysis:** Python-based machine learning and data processing libraries are used for data preprocessing, model training, prediction, and evaluation. The implemented regression algorithms include Linear Regression, Decision Tree Regressor, and Random Forest Regressor.

**Deployment and Development Tools:** Cloudflare Workers is used for frontend deployment, while Render is used for backend deployment. Git and GitHub are used for source code management and version control. Visual Studio Code is used as the primary development environment.

## 3.5 Summary

This chapter presented the system analysis and design of Gradiq – Student Performance Prediction System. It discussed the limitations of traditional academic record management and explained the proposed web-based solution. The technical, operational, and economic feasibility of the project was also examined, followed by the functional and non-functional requirements.

The chapter further described the system architecture, application workflow, input and output design, and technologies used for implementation. The integration of React, Flask, TiDB Cloud, and machine learning components provides a centralized platform for student data management, academic performance analysis, and prediction. The proposed architecture also provides a foundation for future system improvements and additional analytical capabilities.

# 4. METHODOLOGY

## 4.1 Overview

This chapter describes the methodology adopted for developing Gradiq – Student Performance Prediction System. The methodology focuses on managing student academic information, preparing assessment data, analysing academic performance, training machine learning models, and generating performance predictions. The complete process is integrated into a web-based application using a React frontend, Flask backend, and TiDB Cloud database.

The machine learning workflow consists of several stages, including academic data collection, data validation, preprocessing, exploratory data analysis, feature preparation, model training, evaluation, and prediction. Each stage contributes to transforming academic assessment records into meaningful analytical information that can support student performance monitoring and decision-making.

## 4.2 Data Collection

Gradiq uses academic assessment information maintained within its database and machine learning training records. The system is designed to collect and organize student-related academic information through its web-based interface and data management functionalities.

The academic parameters considered by the system include Assignment 1, Assignment 2, Class Test 1, Class Test 2, Presentation Marks, and Attendance. These parameters provide information about a student's performance across different academic activities.

The assessment structure follows the institution's internal evaluation pattern. Assignment 1 and Assignment 2 carry 5 marks each, while Class Test 1 and Class Test 2 carry 20 marks each. The combined class test score is calculated as the average of both class tests and normalized to a maximum of 20 marks. Presentation carries 10 marks. The internal assessment is calculated out of 40 marks, while the End-Semester examination carries 60 marks, making the overall academic score 100 marks.

The collected academic information is stored in the TiDB Cloud database and can be used for academic analysis and machine learning operations, depending on the availability of valid training records and target values.

## 4.3 Data Preprocessing

Data preprocessing is an essential stage of the machine learning workflow because raw academic records may contain missing values, invalid entries, inconsistent formats, or values outside the permitted assessment ranges. Gradiq applies data validation and preprocessing techniques to prepare academic records for analysis and model training.

### 4.3.1 Data Retrieval

The system retrieves the required academic information from the database through backend services. The Flask backend communicates with TiDB Cloud and obtains relevant student and training records for the requested operation. The retrieved information is prepared for further processing by the machine learning components.

### 4.3.2 Data Validation

Data validation ensures that the academic information entered or uploaded into

the system follows the required format and permitted ranges. Assessment marks are checked against their maximum limits, and attendance values are validated according to the expected percentage range. Invalid or incomplete records are identified to maintain data quality and reduce errors during analysis.

### 4.3.3 Handling Missing and Inconsistent Values

Missing and inconsistent values can affect the reliability of machine learning models. Therefore, the preprocessing workflow checks the availability and validity of the required input features and target values. Records that do not satisfy the required training conditions are handled according to the implemented validation rules before model training.

### 4.3.4 Feature Preparation

The academic assessment information is organized into suitable numerical features for machine learning. Assignment marks, class test performance, presentation marks, and attendance are prepared in the format required by the prediction model. Derived assessment values are calculated where necessary to maintain consistency with the academic evaluation structure.

### 4.3.5 Data Preparation

After validation and feature preparation, eligible records are converted into a structured format for exploratory analysis and model training. The prepared dataset is checked to ensure that the required features and corresponding target values are available. Only eligible records are considered for supervised machine learning operations.

## 4.4 Exploratory Data Analysis (EDA)

Exploratory Data Analysis is performed to understand the structure and characteristics of academic data. It helps identify performance patterns, relationships between assessment parameters, and possible data quality issues. Gradiq provides analytical functionalities to examine academic records and present relevant information through visualizations and dashboard components.

### 4.4.1 Statistical Summary

Statistical analysis provides an overview of the academic dataset through numerical measures such as count, mean, minimum, maximum, and distribution of assessment marks. These values help understand the general performance pattern of students and identify unusual or inconsistent observations.

### 4.4.2 Correlation Analysis

Correlation analysis is used to examine the relationship between different

academic parameters. It helps identify whether changes in one assessment component are associated with changes in another academic performance measure. Correlation results can be presented through graphical visualizations to make relationships easier to interpret.

### 4.4.3 Data Visualization

Data visualization presents academic information in an understandable graphical format. Gradiq uses dashboard charts and analytical visualizations to display student performance patterns, assessment summaries, and relevant machine learning results. These visual representations help administrators interpret academic information more efficiently.

## 4.5 Feature Selection and Preparation

Feature selection identifies the input variables that are relevant to the prediction task. In Gradiq, academic assessment parameters are prepared as model input features according to the implemented machine learning pipeline. The selected features must be consistent with the prediction target and should contain sufficient valid information for model training.

The feature preparation process also ensures that the same input structure is maintained during both model training and prediction. This helps avoid inconsistencies between the data used to train the model and the data supplied when generating predictions.

## 4.6 Model Building and Training

Machine learning is an important component of Gradiq. The system uses regression algorithms to analyse academic assessment data and estimate numerical student performance values. The implemented algorithms include Linear Regression, Decision Tree Regressor, and Random Forest Regressor.

Linear Regression is used as a statistical learning approach to model relationships between input features and the target variable. Decision Tree Regressor uses a tree-based structure to identify patterns and generate numerical predictions. Random Forest Regressor combines multiple decision trees to produce predictions based on an ensemble learning approach.

The available training records are divided into training and testing subsets using an 80:20 split. The training subset is used to fit the models, while the testing subset is used to evaluate their performance on unseen records. Model training is performed using eligible records containing valid input features and corresponding actual target values.

## 4.7 Model Evaluation

Model evaluation is performed to measure the predictive performance of the trained regression algorithms. Gradiq uses standard regression evaluation metrics, including Mean Absolute Error (MAE), Mean Squared Error (MSE), Root

Mean Squared Error (RMSE), and $R^2$ Score.

Mean Absolute Error measures the average absolute difference between actual and predicted values. Mean Squared Error calculates the average squared prediction error. Root Mean Squared Error represents the square root of the mean squared error and expresses the prediction error in the same unit as the target variable. $R^2$ Score measures how well the model explains the variation in the target variable.

These evaluation metrics help compare the performance of different regression algorithms and support the selection of a suitable model based on the implemented model selection criteria.

## 4.8 Prediction Process

After model training and evaluation, the selected trained model can be used to generate student performance predictions. The system receives the required academic input values through the application interface and sends them to the Flask backend through API requests. The backend prepares the input features and passes them to the prediction component.

The prediction component processes the supplied academic information and returns the estimated performance value. The result is then communicated to the frontend and displayed through the application interface. When a trained machine learning model is unavailable, the system may provide a separately identified rule-based estimate or indicate that model training is required. A rule-based estimate is not considered an actual machine learning prediction.

## 4.9 Summary

This chapter explained the methodology adopted for developing Gradiq – Student Performance Prediction System. It covered academic data collection, validation, preprocessing, exploratory data analysis, feature preparation, machine learning model training, evaluation, and prediction.

The methodology integrates academic assessment information with machine learning techniques to support student performance analysis. The combination of a web-based interface, backend services, cloud database, and regression models provides a structured approach to academic data management and predictive analysis.

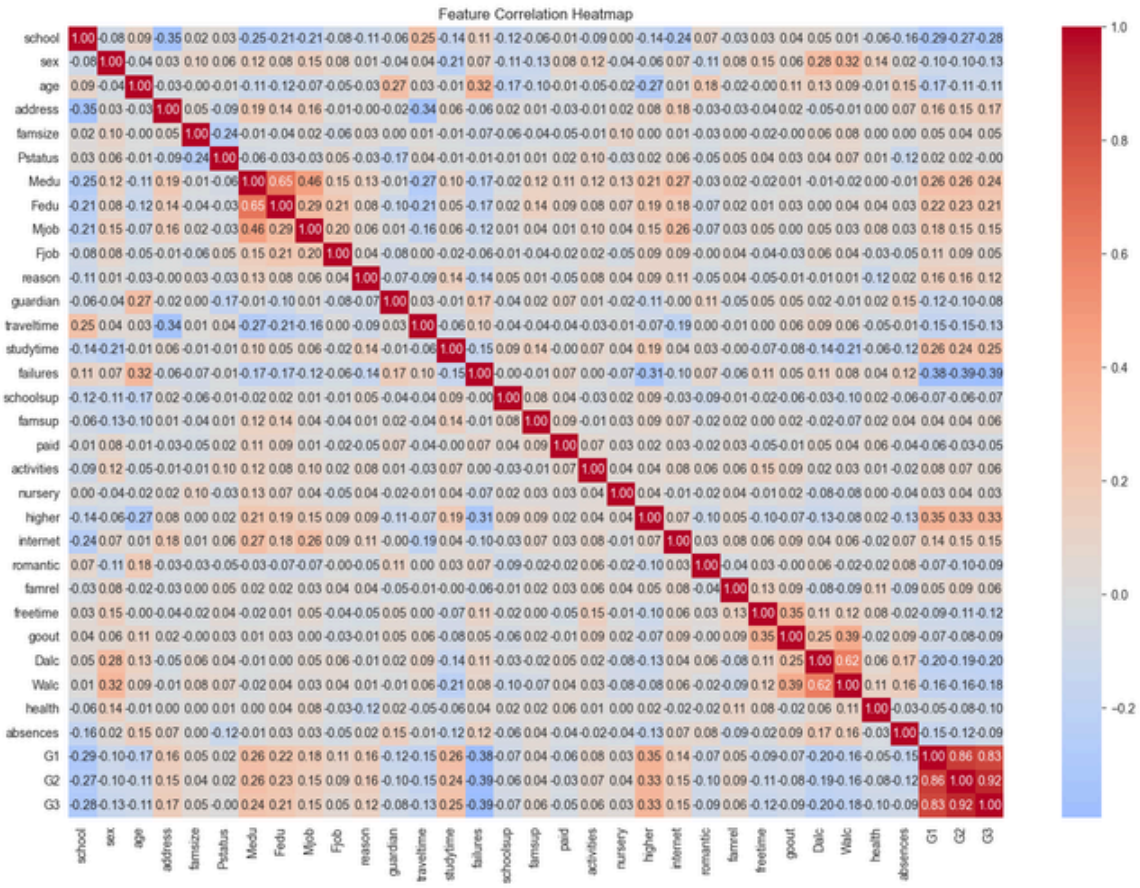



Fig. 4.1

### 4.4.3 Target Distribution

The distribution of the target variable (G3) is plotted using a histogram with a Kernel Density Estimate (KDE) through the plot_target_distribution() function to check its spread and skewness.

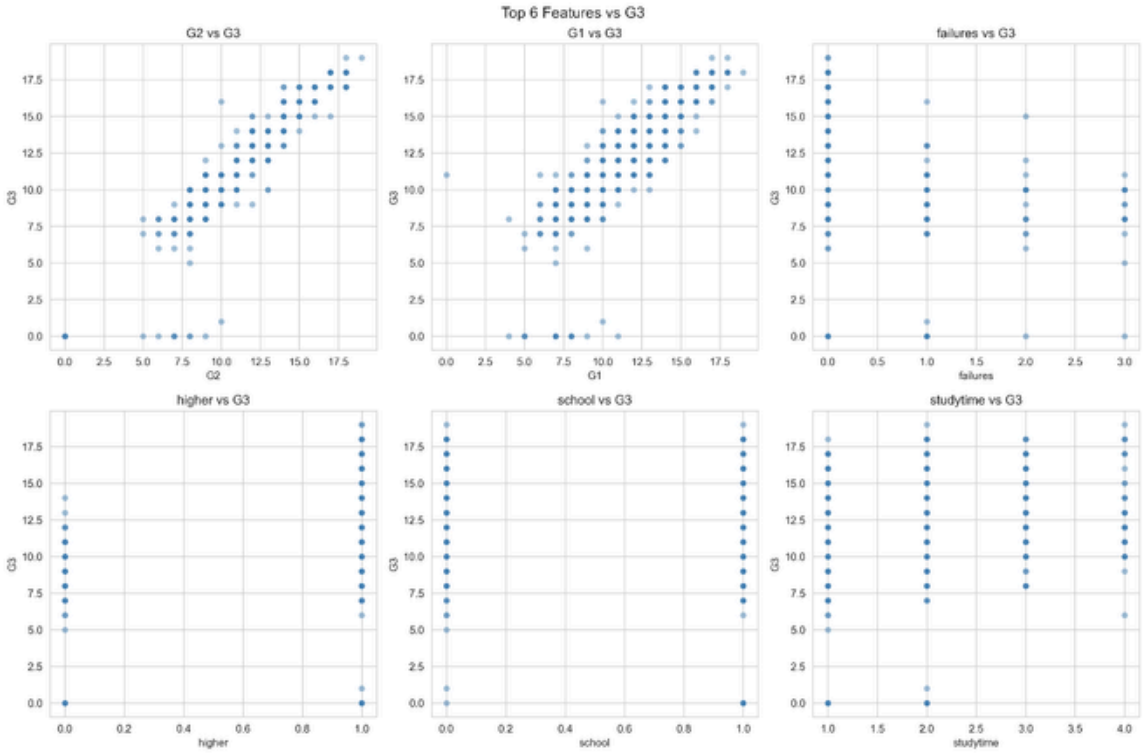



Fig. 4.2

## 4.4.4 Feature Distribution

Feature distribution analysis helps in understanding how academic assessment values are distributed across student records. The distribution of numerical features, such as assignment marks, class test marks, presentation marks, and attendance, can be examined using appropriate statistical summaries and graphical visualizations. This analysis helps identify variations in student performance and unusual data patterns.

## 4.4.5 Academic Performance Analysis

Academic performance analysis examines the relationship between different assessment components and student performance. Assignment marks, class test scores, presentation marks, and attendance are analysed to understand their individual and combined contribution to academic outcomes. Graphical representations help administrators interpret assessment patterns and compare student performance.

## 4.4.6 Relationship Analysis

Relationship analysis is performed to understand the associations between different numerical academic features. Scatter plots and other suitable visualizations can be used to examine relationships between assessment parameters and identify patterns within the academic dataset. These

observations support the selection and preparation of relevant input features for machine learning.

## 4.5 Feature Selection and Preparation

Feature selection and preparation are important stages of the machine learning workflow. The objective is to identify and prepare suitable academic parameters that can be used by the prediction models. Gradiq uses academic assessment information, including assignment marks, class test marks, presentation marks, and attendance, according to the implemented prediction pipeline.

### 4.5.1 Identification of Input Features

The input features are selected based on their relevance to the academic performance prediction task. Assessment-related parameters are organized into a structured numerical format so that they can be processed by the regression algorithms. The selected features must correspond to the input structure expected by the trained model.

### 4.5.2 Feature Preparation Criteria

The selected academic features must contain valid numerical values and follow the permitted assessment ranges. The system prepares the required features consistently during model training and prediction. This ensures that the model receives data in the same structure and format during both operations and reduces the possibility of input-related errors.

## 4.6 Model Building and Evaluation

Model building is an important part of Gradiq because it enables the system to analyse academic assessment data and generate numerical performance predictions. The machine learning pipeline implements multiple regression algorithms and compares their performance to identify a suitable predictive model.

### 4.6.1 Data Splitting

The eligible machine learning dataset is divided into training and testing subsets using the train_test_split() function from Scikit-learn. An 80:20 ratio is used, where 80% of the records are allocated for model training and 20% are reserved for testing. A fixed random state of 42 is used to ensure reproducibility of the data-splitting process.

### 4.6.2 Models Used

Gradiq implements three regression algorithms for academic performance prediction.

Linear Regression: Linear Regression is a statistical machine learning algorithm that models the relationship between input features and a numerical target

variable. It is used as a baseline regression model for estimating academic performance.

Decision Tree Regressor: Decision Tree Regressor uses a tree-based structure to divide data according to feature values and generate numerical predictions. It can capture non-linear relationships between academic assessment parameters and the prediction target.

Random Forest Regressor: Random Forest Regressor is an ensemble learning algorithm that combines multiple decision trees to generate predictions. By aggregating the outputs of several trees, it can capture complex relationships within academic data and help reduce the limitations of an individual decision tree.

### 4.6.3 Model Evaluation Metrics

The performance of the trained regression models is evaluated using standard regression metrics.

Mean Absolute Error (MAE): MAE measures the average absolute difference between actual and predicted values. A lower MAE indicates that the model produces predictions closer to the actual values.

Mean Squared Error (MSE): MSE calculates the average of the squared differences between actual and predicted values. It gives greater importance to larger prediction errors.

Root Mean Squared Error (RMSE): RMSE is calculated by taking the square root of MSE. It represents the prediction error in the same unit as the target variable.

$R^2$ Score (Coefficient of Determination): $R^2$ Score measures how much variation in the target variable is explained by the model. A higher $R^2$ Score generally indicates a better fit on the evaluated dataset.

### 4.6.4 Model Comparison

The implemented regression algorithms are trained using the eligible training records and evaluated using the testing dataset. Their MAE, MSE, RMSE, and $R^2$ Score values are compared to understand their predictive performance. The model selection process uses the evaluation criteria implemented in the backend training pipeline, with lower prediction errors and a higher $R^2$ Score indicating better performance.

The selected model and its evaluation information are stored for use in subsequent prediction operations. The comparison results help identify a suitable model for the available academic training data.

Fig. 4.3: Comparison of Machine Learning Model Performance

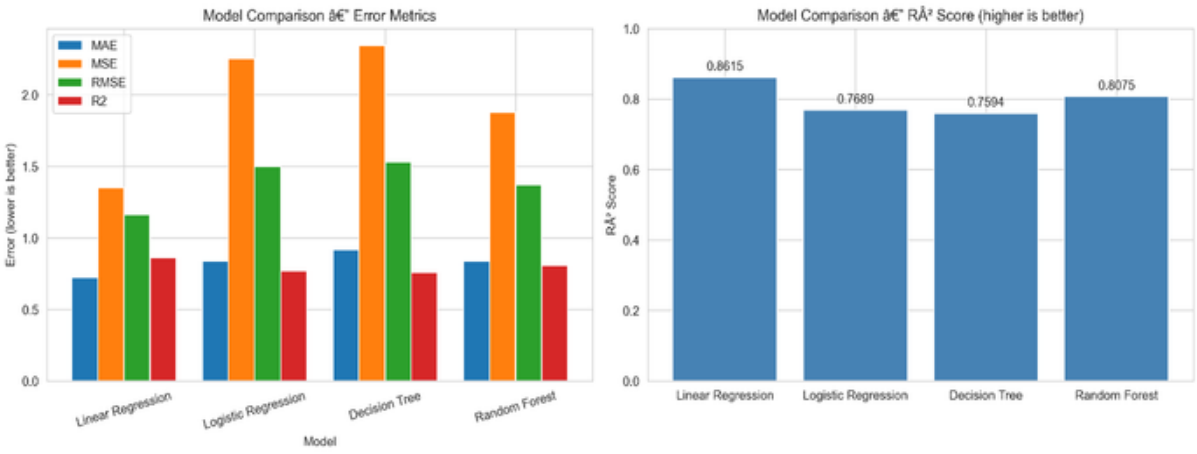



Fig. 4.3

# 4.7 Prediction and Visualization

## 4.7.1 Student Performance Prediction

After model training and evaluation, the selected trained regression model is used to generate student performance predictions. The prediction process is integrated into the Flask backend and is accessed through the application's API.

When a prediction request is received, the backend validates the submitted academic information and prepares the required input features according to the model's expected structure. The trained model processes these features and generates a numerical prediction. The result is then returned to the React frontend and displayed through the application interface.

If a trained machine learning model is not available, the system may provide a separately identified rule-based estimate where supported by the application logic. Such an estimate is not considered a machine learning prediction. The system distinguishes between an ML Prediction, a Rule-Based Estimate, and a Model Not Trained status to provide clarity about the generated result.

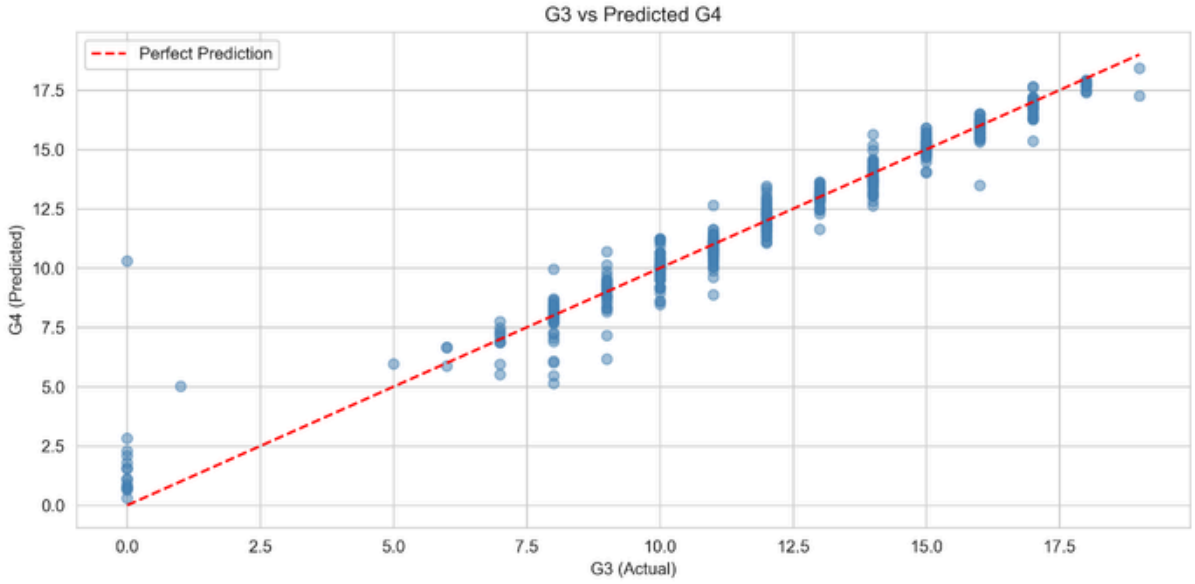



Fig. 4.4

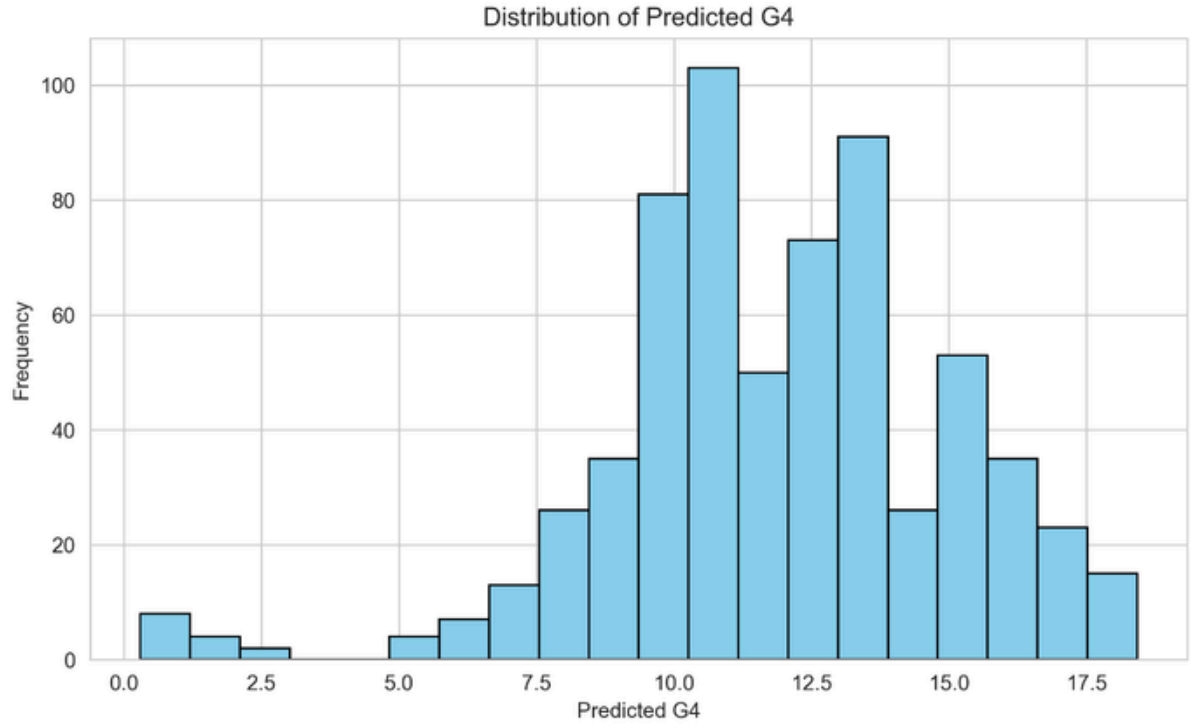



Fig 4.5

## 4.7.2 Visualization of Results

Visualization is an important part of Gradiq because it helps administrators understand

academic performance patterns and interpret machine learning results. The application presents analytical information through interactive dashboards and graphical components. These visualizations make complex academic information easier to understand and support data-driven decision-making.

**Academic Performance Visualization:** The system presents student assessment information, including assignment marks, class test performance, presentation marks, and attendance. These visual representations help administrators understand the academic performance of individual students and identify variations in assessment results.

**Dashboard Analytics:** The dashboard provides summarized academic information through charts, statistics, and performance indicators. It helps administrators monitor student records and understand the overall academic status of the available data.

**Model Comparison Visualization:** The system presents machine learning evaluation results to help compare the performance of different regression algorithms. Evaluation metrics such as MAE, MSE, RMSE, and $R^2$ Score provide information about prediction errors and model performance.

**Exploratory Data Analysis Visualization:** Graphical representations are used to understand the distribution of academic assessment values and relationships between different numerical features. These visualizations help identify patterns and provide insights into the available academic data.

**Prediction Result Visualization:** The application displays generated student performance predictions through its interface. This allows administrators to examine prediction results alongside relevant academic information.

# 4.8 Application Integration and Deployment

Unlike a standalone Python pipeline, Gradiq is developed as an integrated web application consisting of frontend, backend, database, and machine learning components. These components communicate through APIs to provide a complete academic performance management and prediction system.

The React and Vite frontend provides the user interface for authentication, student management, academic record handling, dashboards, and prediction results. It communicates with the Flask backend through REST API requests.

The Flask backend manages application operations, processes incoming requests, performs database interactions, and connects the machine learning components with the frontend. The machine learning module handles academic data preparation, model training, evaluation, and prediction operations.

TiDB Cloud provides centralized database storage for user information, student records, prediction information, and machine learning training records. The frontend application is deployed on Cloudflare Workers, while the backend application is hosted on Render. Git and GitHub are used for source code management and version control.

This integration allows users to access Gradiq through a web browser without manually executing individual Python scripts. It also makes the application easier to maintain, update, and extend.

## 4.9 Summary

This chapter explained the methodology adopted for developing Gradiq – Student Performance Prediction System. It covered academic data collection, preprocessing, exploratory data analysis, feature preparation, machine learning model training, evaluation, prediction, and visualization.

The project integrates multiple regression algorithms, including Linear Regression, Decision Tree Regressor, and Random Forest Regressor, to analyse academic assessment data and generate numerical performance predictions. Standard evaluation metrics are used to compare model performance and assess prediction errors.

The chapter also described the integration of the React frontend, Flask backend, TiDB Cloud database, and machine learning components. The deployed web application provides a centralized environment for managing student academic information, analysing performance patterns, and accessing prediction results. Overall, the methodology establishes the technical foundation for Gradiq and supports its objective of improving academic monitoring through data analysis and machine learning.

# 5. IMPLEMENTATION

## 1. 5.1 Overview

2. This chapter describes the implementation of Gradiq – Student Performance Prediction System. It covers the hardware and software requirements, implementation of different application modules, project execution process, and outputs generated by the system.

3. Gradiq is developed as a web-based application that integrates student information management, academic assessment management, data analysis, and machine learning-based performance prediction. The implementation follows a client-server architecture in which the frontend communicates with the backend through REST APIs, while the backend interacts with the database and machine learning components.

## 1. 5.2 Requirements

### 2. 5.2.1 Hardware Requirements

3. The hardware requirements for developing and running the Gradiq application are as follows.

4. **Processor:** Intel Core i3 or higher

5. **RAM:** 4 GB minimum (8 GB recommended)
6. **Storage:** At least 10 GB of available storage
7. **Input/Output Devices:** Keyboard, Mouse, and Monitor
8. **Internet Connection:** Required for cloud database connectivity and deployed application access

### 9. 5.2.2 Software Requirements

10. The software and technologies used for developing Gradiq include the following.
11. **Operating System:** Windows 10/11 or compatible operating system
12. **Frontend:** React and Vite
13. **Programming Language:** JavaScript and Python
14. **Backend Framework:** Flask
15. **Database:** TiDB Cloud
16. **Machine Learning:** Scikit-learn and supporting Python data-processing libraries used in the implementation
17. **API Communication:** Axios and REST APIs
18. **Data Visualization:** Recharts
19. **Development Environment:** Visual Studio Code
20. **Version Control:** Git and GitHub
21. **Frontend Deployment:** Cloudflare Workers
22. **Backend Deployment:** Render

# 1. 5.3 Module Implementation

2. Gradiq is divided into several functional modules that work together to provide student academic management and performance prediction.
3. **Authentication Module:** This module manages user authentication and provides authorized access to the application. Users can log in through the frontend interface, while the Flask backend handles authentication requests and related operations.

4. **Student Management Module:** This module provides facilities for adding, viewing, updating, and deleting student records. Student information is maintained in the TiDB Cloud database and can be accessed through the application dashboard.

5. **Academic Assessment Module:** This module manages student assessment information, including Assignment 1, Assignment 2, Class Test 1, Class Test 2, Presentation Marks, and Attendance. It supports the organization of academic information according to the defined assessment structure.
6. **Database Management Module:** The database module manages the storage and retrieval of application information. TiDB Cloud is used to store user details, student records, prediction information, and machine learning training records.
7. **Data Validation and Preprocessing Module:** This module prepares academic information for analysis and machine learning operations. It checks the validity of input records, verifies assessment ranges, and prepares eligible numerical features for further processing.

8. **Exploratory Data Analysis Module:** This module analyses available academic records to identify patterns, distributions, and relationships between assessment parameters. The analytical information is presented through graphical visualizations and dashboard components.

9. **Machine Learning Module:** This module implements Linear Regression, Decision Tree Regressor, and Random Forest Regressor for student performance prediction. It handles training data preparation, model training, evaluation, and model selection using appropriate regression metrics.

10. **Prediction Module:** This module receives academic input through backend APIs and uses the available trained model to generate student performance predictions. It also distinguishes machine learning predictions from rule-based estimates when a trained model is unavailable.

11. **Dashboard and Analytics Module:** This module provides an interactive interface for displaying student information, academic summaries, analytical charts, and prediction-related results. It helps administrators monitor academic information from a centralized dashboard.

# 12. 5.4 Execution of the Project

13. Gradiq can be accessed through its deployed web application or executed locally in a development environment.
14. **Local Development Setup:** The project source code is obtained from the GitHub repository and opened in Visual Studio Code. The frontend and backend dependencies are installed separately according to their respective configuration files.

15. **Frontend Execution:** The frontend directory contains the React and Vite application. After installing its dependencies, the development server can be started using the following commands:

```
cd frontend
npm install
npm run dev
```

1. **Backend Execution:** The backend directory contains the Flask application and its Python dependencies. After configuring the required environment variables and database connection, the backend can be started using the project's configured entry point.

```
cd backend
pip install -r requirements.txt
```

1. The Flask application is then started using the appropriate command defined by the backend configuration.
2. **Database Configuration:** The application connects to TiDB Cloud using the database connection settings configured in the backend environment. Required database credentials must be configured securely and should not be included in public source code.

3. **Accessing the Application:** Once the frontend and backend are running and database connectivity is established, the application can be accessed through the frontend development URL. Users can log in and access the available student management, academic assessment, analytics, and prediction functionalities.

# 4. 5.5 Output

5. The Gradiq application generates outputs through its web interface and backend operations. The following screenshots represent the major functional outputs of the system.

6. **5.5.1 Login Page:** The login page provides an authentication interface through which authorized users can access the application.
7. **Fig. 5.1: Gradiq Login Interface**
8. **5.5.2 Admin Dashboard:** The dashboard provides an overview of student

information and academic activities through summary cards, statistics, and graphical components.

9. **Fig. 5.2: Admin Dashboard**
10. **5.5.3 Student Management:** The student management interface allows administrators to view student records and perform operations such as adding, updating, and deleting student information.
11. **Fig. 5.3: Student Management Interface**
12. **5.5.4 Academic Assessment Management:** This interface allows the management of student assessment information, including assignment marks, class test marks, presentation marks, and attendance.
13. **Fig. 5.4: Academic Assessment Interface**
14. **5.5.5 Data Analysis and Visualization:** The analytical interface presents academic data through graphical representations and summaries. It helps administrators understand assessment patterns and student performance.
15. **Fig. 5.5: Academic Data Analysis**
16. **5.5.6 Machine Learning Model Evaluation:** This module displays the evaluation results of the implemented regression models. Metrics such as MAE, MSE, RMSE, and $R^2$ Score are used to compare model performance.
17. **Fig. 5.6: Machine Learning Model Evaluation**
18. **5.5.7 Student Performance Prediction:** The prediction interface displays the generated student performance result based on the available academic information. The system identifies whether the result is generated through a trained machine learning model or a rule-based estimation process.
19. **Fig. 5.7: Student Performance Prediction Result**

## 20. 5.6 Summary

21. This chapter described the implementation of Gradiq – Student Performance Prediction System, including its hardware and software requirements, functional modules, execution process, and application outputs.
22. The system integrates React and Vite for frontend development, Flask for backend operations, and TiDB Cloud for database management. Its machine learning components support academic data processing, regression model training, evaluation, and performance prediction. The application also provides student management, academic assessment handling, and interactive analytical dashboards.

23. The successful integration of these components establishes Gradiq as a centralized web-based platform for managing student academic information and supporting data-driven performance analysis.

# 6. TESTING AND DEBUGGING

## 6.1 Overview

Testing is an essential phase of software development that ensures the application functions according to its specified requirements. It helps identify

errors, verify individual components, and evaluate the overall performance and reliability of the system. Debugging involves identifying, analysing, and correcting errors discovered during development and testing.

In Gradiq – Student Performance Prediction System, testing is performed to verify the functionality of the frontend, backend APIs, database connectivity, student management operations, academic assessment handling, and machine learning components. The testing process focuses on ensuring that the application processes user requests correctly, maintains accurate academic records, and generates prediction results according to the implemented machine learning workflow.

**6.3 Test Result**

| Student_ID | school | sex | age | G1 | G2 | G3 | G4_Predicted |
|---|---|---|---|---|---|---|---|
| 1 | 0 | M | 18 | 0 | 11 | 11 | 10.66 |
| 2 | 0 | M | 17 | 9 | 11 | 11 | 11.01 |
| 3 | 0 | M | 15 | 12 | 13 | 12 | 12.92 |
| 4 | 0 | M | 15 | 14 | 14 | 14 | 14.51 |
| 5 | 0 | M | 16 | 11 | 13 | 13 | 13.03 |
| 6 | 0 | F | 16 | 12 | 12 | 13 | 12.86 |
| 7 | 0 | F | 16 | 13 | 12 | 13 | 12.58 |
| 8 | 0 | M | 17 | 10 | 13 | 13 | 12.91 |
| 9 | 0 | F | 15 | 15 | 16 | 17 | 16.72 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 10 | 0 | F | 15 | 12 | 12 | 13 | 12.86 |

# 6.3 Types of Testing

## 6.3.1 Unit Testing

Unit testing is performed to verify the functionality of individual components of the Gradiq application. Each module is checked separately to ensure that it performs its intended operation correctly.

The Authentication Module is tested to verify login functionality and handling of valid and invalid credentials. The Student Management Module is tested to check the addition, retrieval, updating, and deletion of student records. The Academic Assessment Module is checked to verify the handling and validation of assignment marks, class test marks, presentation marks, and attendance.

The Data Validation and Preprocessing Module is examined to ensure that academic records are checked and prepared correctly before analysis. The Machine Learning Module is tested to verify data preparation, model training, prediction, and calculation of evaluation metrics such as MAE, MSE, RMSE, and $R^2$ Score. The Prediction Module is checked to verify that it returns the appropriate result based on model availability. The Dashboard Module is also tested to ensure that academic information and analytical results are displayed correctly.

## 6.3.2 Integration Testing

Integration testing is performed to verify the communication and data flow between different components of Gradiq. The React frontend communicates with the Flask backend through REST APIs, while the backend interacts with TiDB Cloud for database operations.

This testing checks whether user authentication requests, student management operations, academic assessment information, and machine learning requests are transferred and processed correctly. It also verifies that the backend returns appropriate responses to the frontend and that the application displays the retrieved information correctly.

## 6.3.3 System Testing

System testing evaluates the complete Gradiq application as an integrated system. The testing process covers the major functionalities, starting from user authentication and dashboard access to student management, academic assessment operations, data analysis, and performance prediction.

The application is accessed through its deployed web interface to verify that the frontend, backend, and database work together correctly. This testing helps identify issues that may occur during the interaction of different application components.

## 6.3.4 Validation Testing

Validation testing ensures that the application accepts valid academic information and

handles incorrect or incomplete inputs appropriately. Assessment marks are checked against their permitted maximum values, while attendance is validated according to its expected percentage range.

The machine learning prediction process is also checked to ensure that the required input features are available and follow the expected format. Prediction results generated by a trained model are distinguished from rule-based estimates. A rule-based estimate is not treated as an actual machine learning prediction. Model evaluation results are examined using the available testing records and standard regression metrics.

## 6.4 Debugging

Debugging was performed throughout the development of Gradiq to identify and resolve errors related to frontend configuration, backend API communication, database connectivity, and application deployment.

**Frontend and Backend API Configuration:** During deployment, authentication requests were initially sent to the frontend domain instead of the deployed Flask backend. This resulted in unsuccessful API requests. The issue was resolved by configuring the correct backend API base URL in the frontend production environment.

**Database Connectivity:** Database connection settings were checked to ensure that the Flask backend could communicate with TiDB Cloud and perform the required database operations. Configuration and connectivity issues were addressed during application setup.

**Input Validation:** Academic assessment inputs require appropriate validation to prevent invalid marks and incorrect attendance values from entering the system. Validation logic is used to handle unsuitable input values and maintain data consistency.

**Machine Learning Model Availability:** The prediction workflow requires a trained model and correctly prepared input features. The system handles the situation where a trained model is unavailable by distinguishing a rule-based estimate from an actual machine learning prediction.

**Deployment Configuration:** Frontend and backend deployment configurations were checked to ensure that the application services were accessible through their respective cloud environments. API communication was also verified after deployment.

## 6.5 Test Results

Testing of the Gradiq application focuses on its major functional components, including authentication, student management, academic assessment operations, database connectivity, API communication, and machine learning functionality.

The previously verified application operations include successful access to the deployed dashboard, student record management, and frontend-to-backend API communication after correcting the production API configuration.

The following table summarizes the testing status that should be recorded in the final report after executing the corresponding test cases.

| Test Area | Expected Outcome | Test Status |
|---|---|---|
| User Authentication | Correct handling of login requests | Verified |
| Student Management | Student records can be managed | Verified |
| Frontend-Backend Integration | API requests reach the backend correctly | Verified |
| Database Operations | Records are stored and retrieved correctly | Verify each operation |
| Assessment Validation | Invalid academic inputs are handled | To be verified |
| Machine Learning Training | Eligible records are used for model training | To be verified |
| Model Evaluation | Regression metrics are generated | To be verified |
| Student Prediction | Appropriate prediction result is returned | To be verified |
| Cloud Deployment | Application is accessible through the deployed URL | Verified |

The testing results indicate that the main application functionality and deployment have been checked. Additional verification of machine learning training, prediction, and assessment validation is required before claiming that every test case has passed.

## 6.6 Summary

This chapter discussed the testing and debugging activities performed during the development of Gradiq – Student Performance Prediction System. Unit testing, integration testing, system testing, and validation testing were considered to evaluate the application's individual components and overall functionality.

The debugging process focused on resolving issues related to frontend and backend API communication, database configuration, input validation, machine learning model availability, and cloud deployment. These activities helped improve the reliability and functionality of the application.

The testing process provides a basis for verifying that Gradiq can manage student academic information, communicate with its backend services, and support machine learning-based performance analysis. Further testing with genuine academic training records is necessary to establish the predictive performance of the system.

# 7. CONCLUSION

1. The main objective of this project was to develop **Gradiq – Student Performance Prediction System**, a web-based application designed to simplify student academic record management and support academic performance analysis using machine learning techniques. The system integrates student information management, academic assessment handling, data visualization, and performance prediction into a centralized platform.

2. The project was developed using modern web technologies, including React and Vite for the frontend, Python and Flask for backend development, and TiDB Cloud for database management. The application provides facilities for managing student records, maintaining academic assessment information, and accessing analytical insights through an interactive dashboard. It also integrates machine learning components to process academic data, train regression models, evaluate their performance, and generate student performance predictions.

3. The machine learning pipeline incorporates Linear Regression, Decision Tree Regressor, and Random Forest Regressor to analyse academic assessment data. Model evaluation is supported by standard regression metrics, including Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and $R^2$ Score. These metrics provide a systematic approach to comparing model performance and understanding prediction errors.

4. The project also demonstrates the practical integration of frontend development, backend API services, cloud database management, and machine learning functionality. The frontend is deployed using Cloudflare Workers, while the backend is hosted on Render, with TiDB Cloud providing database services. This deployment architecture makes the application accessible through a web browser and provides a foundation for future improvements.

5. Gradiq aims to help educational administrators organize academic information, monitor student performance, and identify patterns that may indicate the need for additional academic support. Although the system provides a foundation for machine learning-based academic analysis, the reliability of its predictions depends on the availability, quality, and quantity of genuine labelled training data.
6. Overall, Gradiq demonstrates how web technologies and machine learning can be combined to develop a centralized student performance management and prediction platform. The project provides opportunities for further development through larger academic datasets, improved predictive models, advanced analytics, and personalized academic recommendations.

7.

# 8. FUTURE ENHANCEMENT

## 8.1 Overview

Gradiq – Student Performance Prediction System provides a centralized platform for student academic record management, performance analysis, and machine learning-based prediction. Although the current application provides essential functionalities, several improvements can be introduced in the future to enhance its predictive capabilities, scalability, usability, and overall effectiveness.

## 8.2 Proposed Enhancements

**Integration of Larger Academic Datasets:** Future development can include larger and more diverse academic datasets collected from multiple departments, institutions, and academic sessions. A greater volume of genuine and properly labelled training records can help improve model generalization and provide more reliable performance analysis.

**Implementation of Advanced Machine Learning Algorithms:** Additional algorithms such as XGBoost, LightGBM, CatBoost, Support Vector Regression, and Artificial Neural Networks can be explored. Their performance can be compared with the existing regression models to identify suitable approaches for different academic prediction tasks.

**Advanced Feature Engineering:** Additional academic features, such as previous semester performance, subject-wise marks, assessment trends, and attendance patterns, can be incorporated into the machine learning pipeline. These features may provide more detailed information about student academic progress.

**Explainable Artificial Intelligence:** Future versions can integrate explainable AI techniques such as SHAP and feature contribution analysis. This would help administrators understand which academic parameters influenced a particular prediction and improve the interpretability of machine learning results.

**Student Performance Recommendations:** The system can be extended to generate personalized academic recommendations based on student performance patterns. These recommendations may include additional practice, subject-specific improvement suggestions, and academic mentoring.

**Automated Data Integration:** Integration with existing college management systems and Learning Management Systems can automate the collection of student records and assessment information. This can reduce manual data entry and improve the consistency of academic information.

**Mobile Application Development:** A dedicated mobile application can be developed to provide students and administrators with convenient access to academic records, performance analytics, and prediction results through smartphones.

**Advanced Notification and Alert System:** Future development can introduce automated notifications through email, SMS, or in-app alerts to inform students and administrators about academic progress, attendance concerns, and performance-related updates.

**Continuous Model Improvement:** The machine learning pipeline can be enhanced to support periodic model retraining using newly available and verified academic records. This can help maintain model relevance as additional academic data becomes available.

## 8.3 Summary

The proposed future enhancements can extend Gradiq into a more comprehensive academic analytics and decision-support platform. Improvements in dataset quality, predictive modelling, explainability, automated data collection, and personalized recommendations can make the system more useful for educational institutions. These developments can further support informed academic decisions and the identification of students who may require additional academic assistance.

# 9. REFERENCES


Al-Barrak, M. A., & Al-Razgan, M. (2016). Predicting students final GPA using decision trees: A case study. *International Journal of Information and Education Technology, 6*(7), 528–533. https://doi.org/10.7763/IJIET.2016.V6.745

Cortez, P., & Silva, A. M. G. (2008). Using data mining to predict secondary school student performance. In *Proceedings of the 5th Future Business Technology Conference* (pp. 5–12). https://repositorium.sdum.uminho.pt/handle/1822/8024

Flask. (n.d.). *Flask documentation*. https://flask.palletsprojects.com/

GitHub. (n.d.). *GitHub documentation*. https://docs.github.com/

Hussain, S., Dahan, N. A., Ba-Alwib, F. M., & Ribata, N. (2018). Educational data mining and analysis of students' academic performance using WEKA. *Indonesian Journal of Electrical Engineering and Computer Science, 9*(2), 447–459. https://doi.org/10.11591/ijeecs.v9.i2.pp447-459

Kotsiantis, S. B., Pierrakeas, C. J., & Pintelas, P. E. (2004). Predicting students' dropout-prone using machine learning techniques. *Journal of Computer Assisted Learning, 20*(5), 374–383. https://doi.org/10.1111/j.1365-2729.2004.00098.x

McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

NumPy. (n.d.). *NumPy documentation*. https://numpy.org/doc/

Romero, C., & Ventura, S. (2010). Educational data mining: A review of the state of the art. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 40*(6), 601–618. https://doi.org/10.1109/TSMCC.2010.2053532

Scikit-learn. (n.d.). *Scikit-learn documentation*. https://scikit-learn.org/stable/

TiDB. (n.d.). *TiDB documentation*. https://docs.pingcap.com/tidb/stable/

Vite. (n.d.). *Vite documentation*. https://vite.dev/guide/

Meta Open Source. (n.d.). *React documentation*. https://react.dev/

Cloudflare. (n.d.). *Cloudflare Workers documentation*. https://developers.cloudflare.com/workers/

Render. (n.d.). *Render documentation*. https://render.com/docs

Recharts. (n.d.). *Recharts documentation*. https://recharts.org/en-US/